We could “promote” one of its children as the new root (one of them will certainly be next highest priority element: try to prove it as an exercise).

However, we would then only move the problem to the subtree rooted at this node we moved to the root, and then to the next level, and so on.

Instead, since we also have to shrink the array, and since it’s easier to add or remove array’s elements from its tail, we can simply pop the last element in the array and use it to replace the root.

The caveat is that this new root might violate heap’s properties. As a matter of fact, being a leaf, the probability that there is a violation is pretty high. Therefore, we need to reinstate them using a utility method, `pushDown`.

Similarly to what happens for insertion, the nature of the heap and the fact that we move one step towards the leaves at every swap guarantee that no more than a logarithmic number of swaps is required, in the worst case.
In this case as well, concrete implementations have to deal with array sizing, and we can only guarantee an *amortized* logarithmic upper bound (if we need a worst-case logarithmic bound, we need to use static arrays, with all the disadvantages they carry).

Back to our task management example, let’s see what happens calling `top()` on the ternary heap shown at the end